Solution: Same Tree

Problem Statement

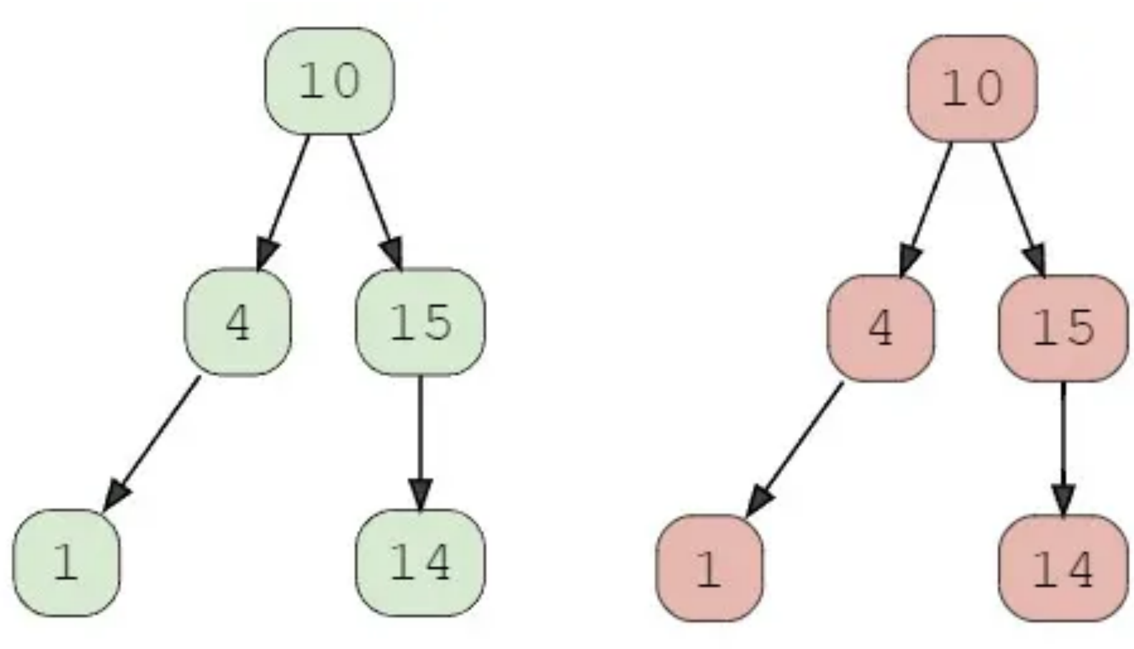
Given the roots of two binary trees 'p' and 'q', write a function to check if they are the same or not.

Two binary trees are considered the same if they met following two conditions:

1. Both tree are structurally identical.
2. Each corresponding node on both the trees have the same value.

Example 1:

Given the following two binary trees:

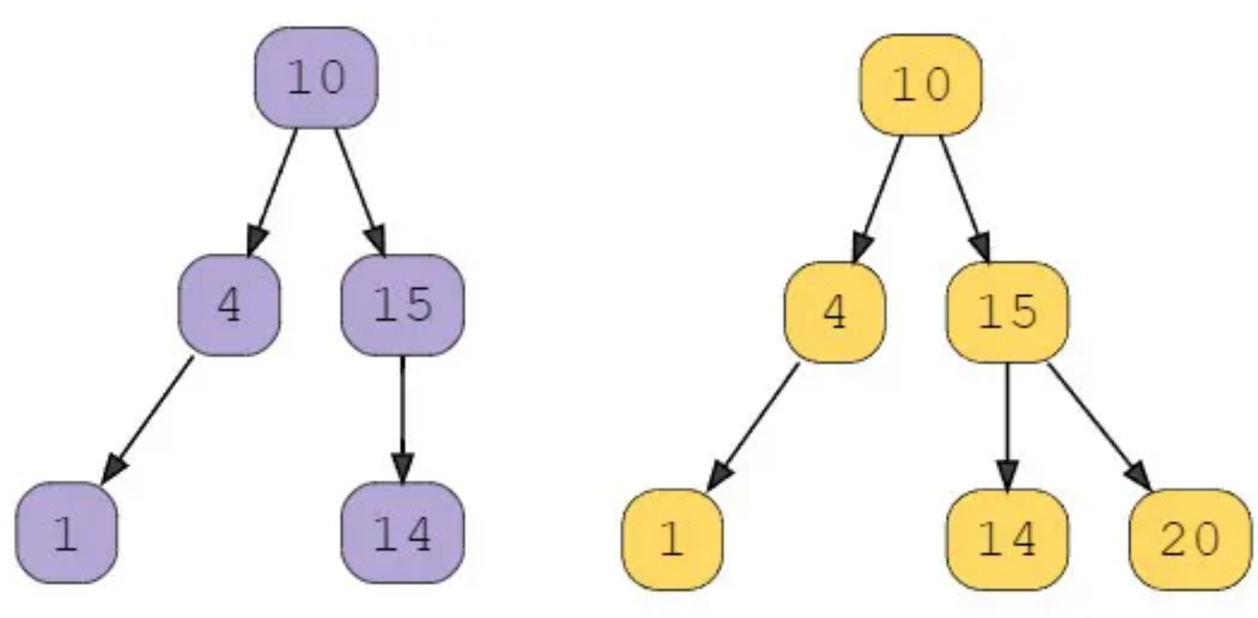


Output: true

Explanation: Both trees are structurally identical and have same values.

Example 2:

Given the following two binary trees:

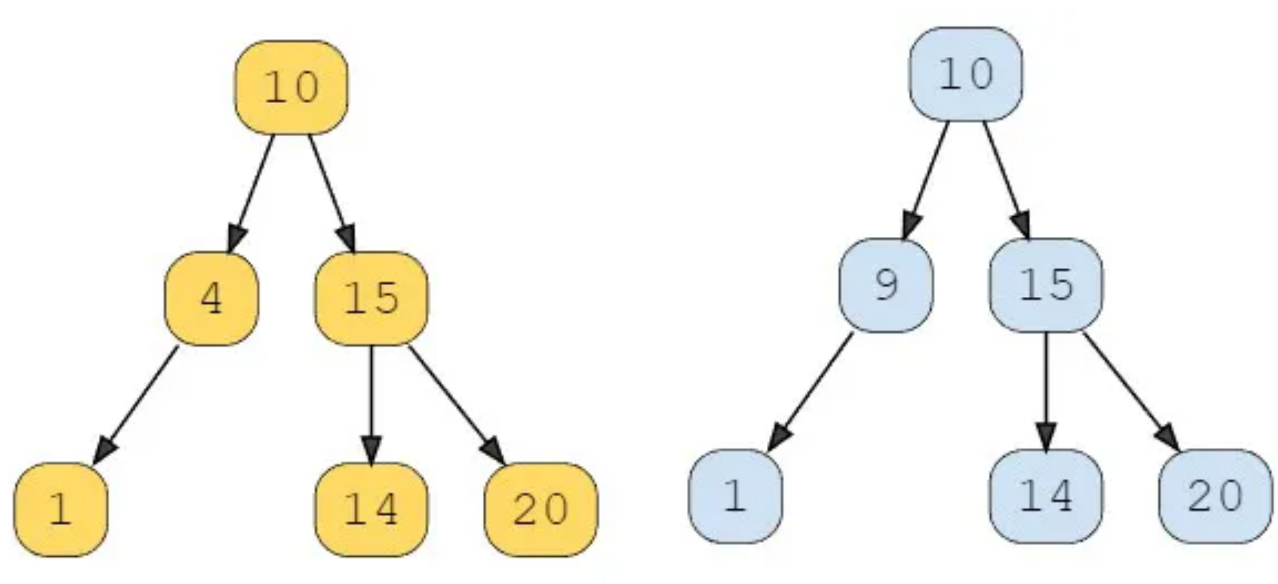


Output: false

Explanation: Trees are structurally different.

Example 3:

Given the following two binary trees:



Output: false

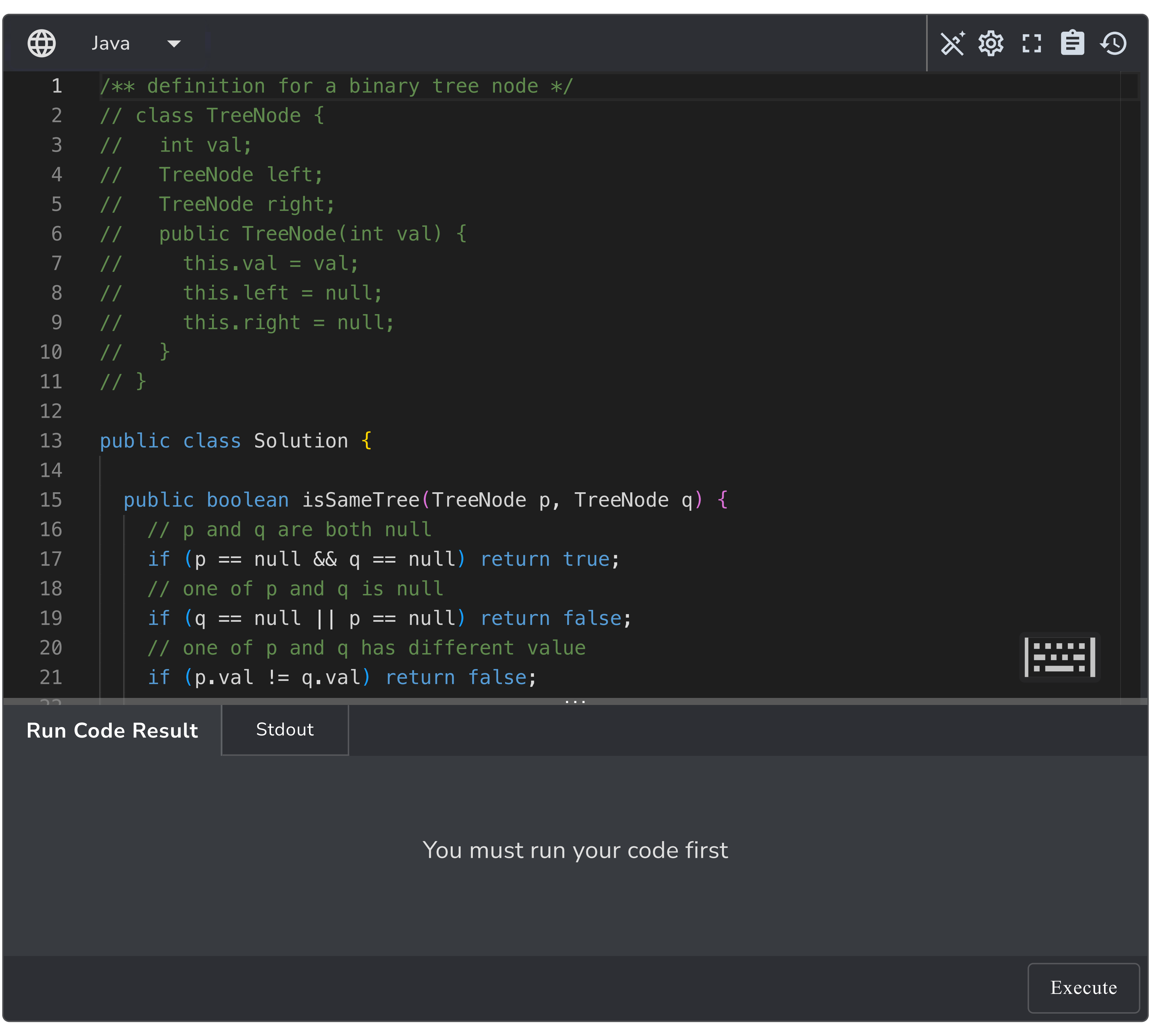
Explanation: Corresponding nodes have different value (4 & 9).

Solution

A simple solution would be to recursively check each corresponding node of the two trees. If one of the trees do not have a corresponding node or their values differ, we can conclude that the trees are not the same.

Code

Here is what our algorithm will look like:



```
/** definition for a binary tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {

  public boolean isSameTree(TreeNode p, TreeNode q) {
    // p and q are both null
    if (p == null && q == null) return true;
    // one of p and q is null
    if (q == null || p == null) return false;
    // one of p and q has different value
    if (p.val != q.val) return false;
```

You must run your code first

Time Complexity

The solution will take O(min(M, N)) time, where 'M' and 'N' are the number of nodes in the given trees respectively. We are taking minimum of 'M' and 'N', since as soon as we see a difference in value or structure, we do not check the remaining trees.

Space Complexity

We will need O(N) space for the recursion stack in the worst case (when the binary tree is a list). Overall, we will need to store nodes equal to the height of the tree, hence, O(H) space complexity, where H is the height of the given tree.

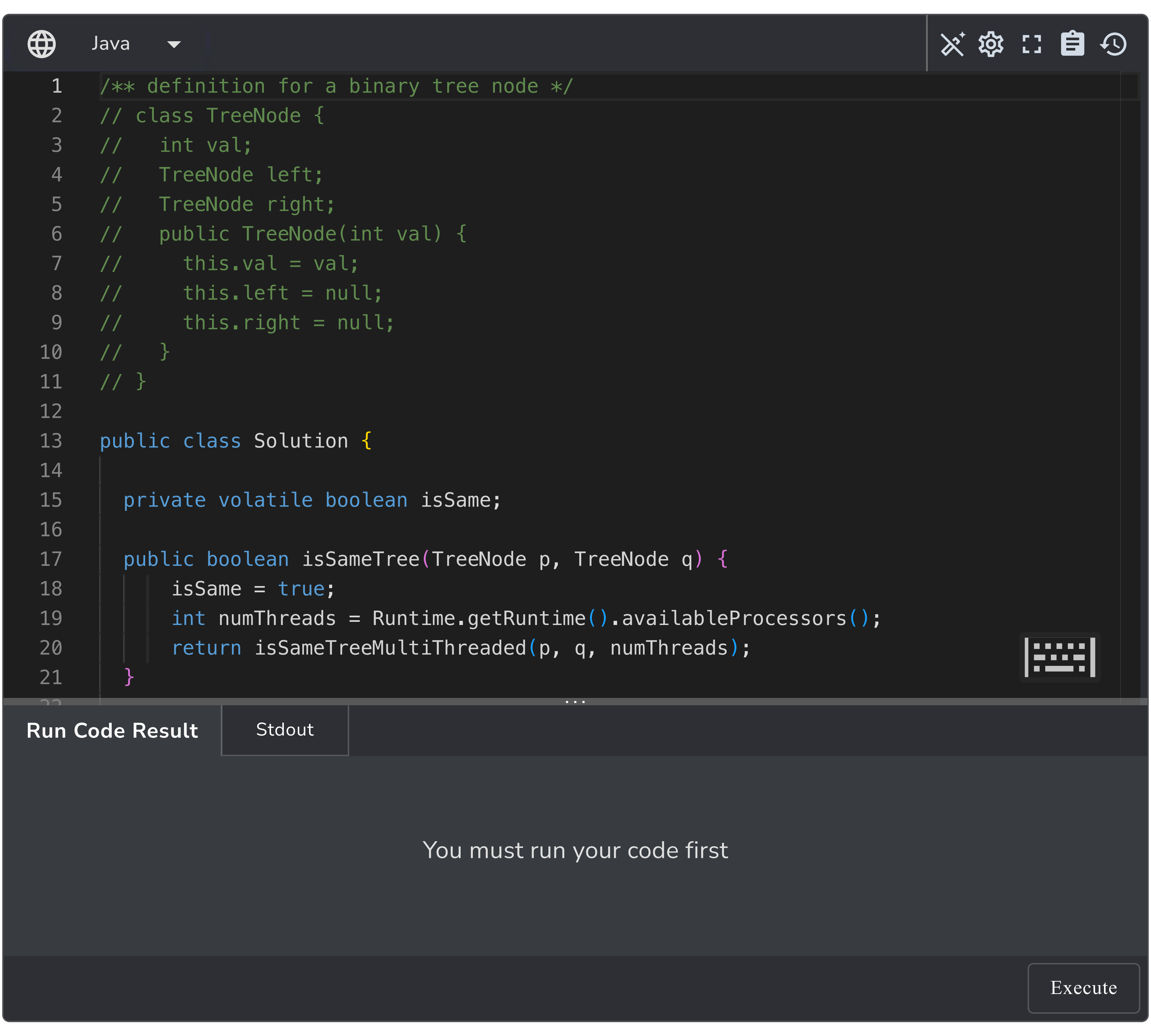
Making the Algorithm Multi-threaded

To further improve the algorithm, we can make **isSameTree**() multi-threaded to check the left and right subtrees in separate threads.

We can find how many processors the machine has on which our algorithm is running. We will, then, start multiple threads so that each core can run one thread.

We will use a Volatile Boolean variable **isSame** so that multiple threads can update its value concurrently.

Here is the code that takes care of this scenario:



```
/** definition for a binary tree node */
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   public TreeNode(int val) {
//     this.val = val;
//     this.left = null;
//     this.right = null;
//   }
// }

public class Solution {

  private volatile boolean isSame;

  public boolean isSameTree(TreeNode p, TreeNode q) {
    isSame = true;
    int numThreads = Runtime.getRuntime().availableProcessors();
    return isSameTreeMultiThreaded(p, q, numThreads);
  }
```

You must run your code first

Time and Space Complexities

Everything has the same complexity as the previous solution.

← Previous
Same Tree (Medium)

Next →
Invert Binary Tree (Medium)

☐ Mark as Completed